

Started on	Thursday, 19 June 2025, 12:48 AM
State	Finished
Completed on	Thursday, 19 June 2025, 1:00 AM
Time taken	12 mins 33 secs
Marks	2.00/2.00
Grade	10.00 out of 10.00 (100%)

Viết chương trình đọc một **đơn đồ thị có hướng, có trọng số không âm** từ bàn phím. Cài đặt thuật toán Moore – Dijkstra để tìm (các) đường đi ngắn nhất từ đỉnh 1 đến các đỉnh còn lại. In các thông tin $pi[u]$ và $p[u]$ của các đỉnh ra màn hình.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m ($1 \leq n < 100$; $0 \leq m < 500$)
- m dòng tiếp theo mỗi dòng chứa 3 số nguyên u, v, w mô tả cung (u, v) có trọng số w ($0 \leq w \leq 100$).

Đầu ra (Output)

- In ra màn hình $pi[u]$ và $p[u]$ của các đỉnh theo mẫu:

```
pi[1] = 0, pi[1] = -1
pi[2] = 3, pi[2] = 1
...
```

Xem thêm ví dụ bên dưới.

Chú ý

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng `freopen("dt.txt", "r", stdin);` vào ngay sau hàm `main()`. Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d%d", &u, &v, &w);
    add_edge(&G, u, v, w);
}
```

For example:

Input	Result
3 4	$pi[1] = 0, p[1] = -1$
1 2 9	$pi[2] = 4, p[2] = 3$
2 3 5	$pi[3] = 4, p[3] = 1$
1 3 4	
3 2 0	

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 // Bước 1: Biểu diễn bằng phương pháp ma trận trọng số (adjacency matrix)
4 #define NO_EDGE -1 // Không có cung giữa hai đỉnh
5 #define MAX_N 101 // Tối đa 100 đỉnh, đánh số đỉnh từ 1
6 typedef struct{ // Khai báo struct kiểu C++, không cần dùng typedef
7     int n, m; // n: số đỉnh, m: số cung
8     int W[MAX_N][MAX_N]; // Ma trận trọng số: W[u][v] == edge weight (u, v)
9 }Graph;
10
11 // Bước 2: Khai báo các hằng và biến hỗ trợ (biến toàn cục):
12 #define NOT_SURE 0 // Chưa chắc chắn
13 #define SURE 1 // Chắc chắn
14 #define oo 99999999 // Xem như vô cùng
15
16 int mark[MAX_N]; // Trạng thái đánh dấu của các đỉnh
17 int d[MAX_N]; // Chiều dài đường đi ngắn nhất
18 int parent[MAX_N];
19
20 // Bước 3: Cài đặt Dijkstra tìm đường đi ngắn nhất từ đỉnh s
21 void Moore_Dijkstra(Graph* pG, int s) {
22     // Đánh dấu tất cả các đỉnh đều NOT_SURE
23     for (int i = 1; i <= pG->n; i++) mark[i] = NOT_SURE;
24 }
```

```

25 // Gán d[v] = oo với mọi v trong V
26 for (int i = 1; i <= pG->n; i++) d[i] = oo;
27
28 // Gán d[s] = 0
29 d[s] = 0;
30
31 // Lặp n lần (mỗi lần chọn một đỉnh và đánh dấu nó là SURE)
32 for (int it = 1; it <= pG->n; it++) {
33     // Tìm đỉnh u NOT_SURE có d[u] nhỏ nhất
34     int u = -1;
35     int min_d = oo;
36     for (int v = 1; v <= pG->n; v++)
37         // Nếu v NOT_SURE
38         if (mark[v] == NOT_SURE)
39             if (d[v] < min_d) {
40                 // Gán min_d = d[v]
41                 min_d = d[v];
42
43                 u = v; // Cho u = v
44             }
45
46     if (u == -1) // Không tìm được u
47         break;
48
49     // Xét các đỉnh kề v của u và cập nhật d[v]
50     for (int v = 1; v <= pG->n; v++)
51         // Nếu u kề với v
52         if (pG->W[u][v] != NO_EDGE)
53             // Nếu d[v] lớn hơn d[u] + edgeWeight(u, v)
54             if (d[v] > d[u] + pG->W[u][v])
55                 // Cập nhật d[v] = d[u] + edgeWeight(u, v)
56                 {
57                     d[v] = d[u] + pG->W[u][v];
58                     parent[v] = u;
59                 }
60
61     // Đánh dấu u là SURE
62     mark[u] = SURE;
63 }
64 }
65
66 // Bước 4: Kiểm thử
67 int main() {
68     Graph G;
69
70     scanf("%d %d", &G.n, &G.m);
71     for (int u = 1; u <= G.n; u++)
72         for (int v = 1; v <= G.n; v++)
73             // Gán weight(u, v) = NO_EDGE
74             G.W[u][v] = NO_EDGE;
75
76     // Đọc các cung và thêm vào đồ thị
77     for (int e = 1; e <= G.m; e++) {
78         int u, v, w;
79         scanf("%d %d %d", &u, &v, &w);
80
81         // Gán trọng số cung (u, v) là w
82         G.W[u][v] = w;
83
84         // Gán weight(v, u) = w (Giả sử G là đồ thị vô hướng)
85         G.W[v][u] = w;
86     }
87
88     for (int u = 1; u <= G.n; u++){
89         parent[u] = -1;
90     }
91
92     Moore_Dijkstra(&G, 1);
93
94     for (int u = 1; u <= G.n; u++){
95         printf("pi[%d] = %d, p[%d] = %d\n", u, d[u], u, parent[u]);
96     }
97
98     return 0;
99 }
100

```

	Input	Expected	Got	
✓	3 4 1 2 9 2 3 5 1 3 4 3 2 0	pi[1] = 0, p[1] = -1 pi[2] = 4, p[2] = 3 pi[3] = 4, p[3] = 1	pi[1] = 0, p[1] = -1 pi[2] = 4, p[2] = 3 pi[3] = 4, p[3] = 1	✓
✓	3 4 1 2 5 2 3 1 1 3 10 3 1 4	pi[1] = 0, p[1] = -1 pi[2] = 5, p[2] = 1 pi[3] = 6, p[3] = 2	pi[1] = 0, p[1] = -1 pi[2] = 5, p[2] = 1 pi[3] = 6, p[3] = 2	✓
✓	6 9 1 2 7 1 3 9 1 5 14 2 3 10 2 4 15 3 4 11 3 5 2 4 6 6 5 6 9	pi[1] = 0, p[1] = -1 pi[2] = 7, p[2] = 1 pi[3] = 9, p[3] = 1 pi[4] = 20, p[4] = 3 pi[5] = 11, p[5] = 3 pi[6] = 20, p[6] = 5	pi[1] = 0, p[1] = -1 pi[2] = 7, p[2] = 1 pi[3] = 9, p[3] = 1 pi[4] = 20, p[4] = 3 pi[5] = 11, p[5] = 3 pi[6] = 20, p[6] = 5	✓

Passed all tests! ✓

Question author's solution (C):

```

1 #include <stdio.h>
2
3 #define MAXN 100
4 #define oo 999999
5 #define NO_EDGE -1
6
7
8 typedef struct {
9     int n, m;
10    int W[MAXN][MAXN];
11 } Graph;
12
13 void init_graph(Graph *pG, int n) {
14     pG->n = n;
15     pG->m = 0;
16     for (int u = 1; u <= n; u++)
17         for (int v = 1; v <= n; v++)
18             pG->W[u][v] = NO_EDGE;
19 }
20
21 void add_edge(Graph *pG, int u, int v, int w) {
22     pG->W[u][v] = w;
23 }
24
25 int mark[MAXN];
26 int pi[MAXN];
27 int p[MAXN];
28
29 void MooreDijkstra(Graph *pG, int s) {
30     int u, v, it;
31     for (u = 1; u <= pG->n; u++) {
32         pi[u] = oo;
33         mark[u] = 0;
34     }
35
36     pi[s] = 0; //chiều dài đường đi ngắn nhất từ s đến chính nó bằng 0
37     p[s] = -1; //trước đỉnh s không có đỉnh nào cả
38
39     //Lặp n-1 lần
40     for (it = 1; it < pG->n; it++) {
41         //1. Tìm u có mark[u] == 0 và có pi[u] nhỏ nhất
42         int j, min_pi = oo;
43         for (j = 1; j <= pG->n; j++)
44             if (mark[j] == 0 && pi[j] < min_pi) {

```

```

44     1 (mark[j] == 0 && pi[j] < min_pi) {
45         min_pi = pi[j];
46         u = j;
47     }
48     //2. Đánh dấu u đã xét
49     mark[u] = 1;
50
51     //3. Cập nhật pi và p của các đỉnh kề của v (nếu thoà)
52     for (v = 1; v <= pG->n; v++)
53         if (pG->W[u][v] != NO_EDGE && mark[v] == 0)
54             if (pi[u] + pG->W[u][v] < pi[v]) {
55                 pi[v] = pi[u] + pG->W[u][v];
56                 p[v] = u;
57             }
58     }
59 }
60
61
62 int main() {
63     Graph G;
64     int n, m;
65     scanf("%d%d", &n, &m);
66     init_graph(&G, n);
67
68     for (int e = 0; e < m; e++) {
69         int u, v, w;
70         scanf("%d%d%d", &u, &v, &w);
71         add_edge(&G, u, v, w);
72     }
73
74     MooreDijkstra(&G, 1);
75     for (int u = 1; u <= G.n; u++)
76         printf("pi[%d] = %d, p[%d] = %d\n", u, pi[u], u, p[u]);
77
78     return 0;
79 }

```

Correct

Marks for this submission: 1.00/1.00.

Viết chương trình đọc một **đơn đồ thị vô hướng, có trọng số không âm** từ bàn phím. Cài đặt thuật toán Moore – Dijkstra để tìm (các) đường đi ngắn nhất từ đỉnh 1 đến các đỉnh còn lại. In các thông tin $pi[u]$ và $p[u]$ của các đỉnh ra màn hình.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m ($1 \leq n < 100$; $0 \leq m < 500$)
- m dòng tiếp theo mỗi dòng chứa 3 số nguyên u, v, w mô tả cung (u, v) có trọng số w ($0 \leq w \leq 100$).

Đầu ra (Output)

- In ra màn hình $pi[u]$ và $p[u]$ của các đỉnh theo mẫu:

```
pi[1] = 0, pi[1] = -1
pi[2] = 3, pi[2] = 1
...
```

Xem thêm ví dụ bên dưới.

Chú ý

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng `freopen("dt.txt", "r", stdin);` vào ngay sau hàm `main()`. Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d%d", &u, &v, &w);
    add_edge(&G, u, v, w);
}
```

For example:

Input	Result
4 5	$pi[1] = 0, p[1] = -1$
1 2 10	$pi[2] = 9, p[2] = 4$
3 2 6	$pi[3] = 4, p[3] = 1$
3 1 4	$pi[4] = 4, p[4] = 3$
3 4 0	
4 2 5	

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 // Bước 1: Biểu diễn bằng phương pháp ma trận trọng số (adjacency matrix)
4 #define NO_EDGE -1 // Không có cung giữa hai đỉnh
5 #define MAX_N 101 // Tối đa 100 đỉnh, đánh số đỉnh từ 1
6 typedef struct{ // Khai báo struct kiểu C++, không cần dùng typedef
7     int n, m; // n: số đỉnh, m: số cung
8     int W[MAX_N][MAX_N]; // Ma trận trọng số: W[u][v] == edge weight (u, v)
9 }Graph;
10
11 // Bước 2: Khai báo các hằng và biến hỗ trợ (biến toàn cục):
12 #define NOT_SURE 0 // Chưa chắc chắn
13 #define SURE 1 // Chắc chắn
14 #define oo 99999999 // Xem như vô cùng
15
16 int mark[MAX_N]; // Trạng thái đánh dấu của các đỉnh
17 int d[MAX_N]; // Chiều dài đường đi ngắn nhất
18 int parent[MAX_N];
19
20 // Bước 3: Cài đặt Dijkstra tìm đường đi ngắn nhất từ đỉnh s
21 void Moore_Dijkstra(Graph* pG, int s) {
22     // Đánh dấu tất cả các đỉnh đều NOT_SURE
23     for (int i = 1; i <= pG->n; i++) mark[i] = NOT_SURE;
```

```

23 for (int i = 1; i <= pG->n; i++) mark[i] = NOT_SURE;
24
25 // Gán d[v] = oo với mọi v trong V
26 for (int i = 1; i <= pG->n; i++) d[i] = oo;
27
28 // Gán d[s] = 0
29 d[s] = 0;
30
31 // Lặp n lần (mỗi lần chọn một đỉnh và đánh dấu nó là SURE)
32 for (int it = 1; it <= pG->n; it++) {
33     // Tìm đỉnh u NOT_SURE có d[u] nhỏ nhất
34     int u = -1;
35     int min_d = oo;
36     for (int v = 1; v <= pG->n; v++)
37         // Nếu v NOT_SURE
38         if (mark[v] == NOT_SURE)
39             if (d[v] < min_d) {
40                 // Gán min_d = d[v]
41                 min_d = d[v];
42
43                 u = v; // Cho u = v
44             }
45
46     if (u == -1) // Không tìm được u
47         break;
48
49     // Xét các đỉnh kề v của u và cập nhật d[v]
50     for (int v = 1; v <= pG->n; v++)
51         // Nếu u kề với v
52         if (pG->W[u][v] != NO_EDGE)
53             // Nếu d[v] lớn hơn d[u] + edgeWeight(u, v)
54             if (d[v] > d[u] + pG->W[u][v])
55                 // Cập nhật d[v] = d[u] + edgeWeight(u, v)
56                 {
57                     d[v] = d[u] + pG->W[u][v];
58                     parent[v] = u;
59                 }
60
61     // Đánh dấu u là SURE
62     mark[u] = SURE;
63 }
64 }
65
66 // Bước 4: Kiểm thử
67 int main() {
68     Graph G;
69
70     scanf("%d %d", &G.n, &G.m);
71     for (int u = 1; u <= G.n; u++)
72         for (int v = 1; v <= G.n; v++)
73             // Gán weight(u, v) = NO_EDGE
74             G.W[u][v] = NO_EDGE;
75
76     // Đọc các cung và thêm vào đồ thị
77     for (int e = 1; e <= G.m; e++) {
78         int u, v, w;
79         scanf("%d %d %d", &u, &v, &w);
80
81         // Gán trọng số cung (u, v) là w
82         G.W[u][v] = w;
83
84         // Gán weight(v, u) = w (Giả sử G là đồ thị vô hướng)
85         G.W[v][u] = w;
86     }
87
88     for (int u = 1; u <= G.n; u++){
89         parent[u] = -1;
90     }
91
92     Moore_Dijkstra(&G, 1);
93
94     for (int u = 1; u <= G.n; u++){
95         printf("pi[%d] = %d, p[%d] = %d\n", u, d[u], u, parent[u]);
96     }
97
98     return 0;
99 }
100

```

	Input	Expected	Got	
✓	4 5 1 2 10 3 2 6 3 1 4 3 4 0 4 2 5	pi[1] = 0, p[1] = -1 pi[2] = 9, p[2] = 4 pi[3] = 4, p[3] = 1 pi[4] = 4, p[4] = 3	pi[1] = 0, p[1] = -1 pi[2] = 9, p[2] = 4 pi[3] = 4, p[3] = 1 pi[4] = 4, p[4] = 3	✓
✓	3 3 3 1 8 1 2 4 3 2 1	pi[1] = 0, p[1] = -1 pi[2] = 4, p[2] = 1 pi[3] = 5, p[3] = 2	pi[1] = 0, p[1] = -1 pi[2] = 4, p[2] = 1 pi[3] = 5, p[3] = 2	✓
✓	6 9 1 2 7 1 3 9 5 1 14 2 3 10 4 4 15 3 4 11 3 5 2 4 6 6 6 5 12	pi[1] = 0, p[1] = -1 pi[2] = 7, p[2] = 1 pi[3] = 9, p[3] = 1 pi[4] = 20, p[4] = 3 pi[5] = 11, p[5] = 3 pi[6] = 23, p[6] = 5	pi[1] = 0, p[1] = -1 pi[2] = 7, p[2] = 1 pi[3] = 9, p[3] = 1 pi[4] = 20, p[4] = 3 pi[5] = 11, p[5] = 3 pi[6] = 23, p[6] = 5	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  #define MAXN 100
4  #define oo 999999
5  #define NO_EDGE -1
6
7
8  typedef struct {
9      int n, m;
10     int W[MAXN][MAXN];
11 } Graph;
12
13 void init_graph(Graph *pG, int n) {
14     pG->n = n;
15     pG->m = 0;
16     for (int u = 1; u <= n; u++)
17         for (int v = 1; v <= n; v++)
18             pG->W[u][v] = NO_EDGE;
19 }
20
21 void add_edge(Graph *pG, int u, int v, int w) {
22     pG->W[u][v] = w;
23     pG->W[v][u] = w;
24     pG->m++;
25 }
26
27 int mark[MAXN];
28 int pi[MAXN];
29 int p[MAXN];
30
31 void MooreDijkstra(Graph *pG, int s) {
32     int u, v, it;
33     for (u = 1; u <= pG->n; u++) {
34         pi[u] = oo;
35         mark[u] = 0;
36     }
37
38     pi[s] = 0; //chiều dài đường đi ngắn nhất từ s đến chính nó bằng 0
39     p[s] = -1; //trước đỉnh s không có đỉnh nào cả
40
41     //Lặp n-1 lần
42     for (it = 1; it < pG->n; it++) {
43         //1. Tìm u có mark[u] == 0 và có pi[u] nhỏ nhất

```



```

44     int j, min_pi = oo;
45     for (j = 1; j <= pG->n; j++)
46         if (mark[j] == 0 && pi[j] < min_pi) {
47             min_pi = pi[j];
48             u = j;
49         }
50     //2. Đánh dấu u đã xét
51     mark[u] = 1;
52
53     //3. Cập nhật pi và p của các đỉnh kề của v (nếu thoả)
54     for (v = 1; v <= pG->n; v++)
55         if (pG->W[u][v] != NO_EDGE && mark[v] == 0)
56             if (pi[u] + pG->W[u][v] < pi[v]) {
57                 pi[v] = pi[u] + pG->W[u][v];
58                 p[v] = u;
59             }
60     }
61 }
62
63
64 int main() {
65     Graph G;
66     int n, m;
67     scanf("%d%d", &n, &m);
68     init_graph(&G, n);
69
70     for (int e = 0; e < m; e++) {
71         int u, v, w;
72         scanf("%d%d%d", &u, &v, &w);
73         add_edge(&G, u, v, w);
74     }
75
76     MooreDijkstra(&G, 1);
77     for (int u = 1; u <= G.n; u++)
78         printf("pi[%d] = %d, p[%d] = %d\n", u, pi[u], u, p[u]);
79
80     return 0;
81 }

```

Correct

Marks for this submission: 1.00/1.00.

◀ * 7. Moore - Dijkstra trực tiếp trên đồ thị (random)

Jump to...

Bài tập 2 - Thuật toán Moore - Dijkstra (chiều dài) ▶

